

实验一 A + B 综合实验报告

一、环境信息表

项目	版本/信息
操作系统	Microsoft Windows 11 家庭版 中文版 (10.0.26200)
TRAE 版本	Trae IDE v2.x (智能体驱动开发环境)
Python 版本	Python 3.12.10
Node.js 版本	未安装 (本实验未涉及)
项目目录	e:\Trae CN\zhihuinyinhangshiyang wxn
实验日期	2026年6月24日

二、TRAE 安装配置过程

2.1 安装步骤

- 下载安装包**: 从官方渠道下载 Trae IDE 安装程序
- 运行安装向导**: 双击安装包, 按照提示完成安装
- 配置工作目录**: 设置工作目录为 e:\Trae CN\zhihuinyinhangshiyang wxn
- 安装依赖模块**: 通过内置终端安装 pandas、openpyxl 等 Python 库

2.2 配置截图说明

(由于文档限制, 以下为截图内容描述)

截图1 - 安装界面: 显示 Trae IDE 安装向导进度条, 正在复制文件到系统目录

截图2 - 欢迎界面: Trae IDE 启动后的欢迎页面, 显示最近项目列表和快捷操作入口

截图3 - 工作区配置: 显示工作区设置界面, 已配置工作目录路径

截图4 - 终端配置：显示内置终端已成功配置 Python 环境

三、贪吃蛇游戏实验

3.1 游戏运行截图说明

截图1 - 游戏初始界面：600×600 画布，蓝色蛇头位于中心，绿色蛇身初始长度为3节，红色食物随机分布

截图2 - 游戏进行中：蛇正在移动，分数显示为"当前分数: 5 | 最高分: 12"

截图3 - 游戏结束界面：显示"游戏结束！分数: 8"，并有"再来一局"按钮

截图4 - 最高分记录：显示最高分已更新为15，数据存储在 localStorage 中

3.2 关键提示词自评

提示词要素	自评
清晰度	高 - 需求描述清晰，包含画布尺寸、颜色、功能等具体要求
完整性	高 - 涵盖了游戏核心功能（移动、吃食物、计分、游戏结束）
可执行性	高 - 技术要求明确（纯HTML+CSS+JS，不使用第三方库）
细节丰富度	中高 - 指定了颜色、控制方式、数据持久化方式
约束条件	中 - 明确要求单文件实现

3.3 游戏功能验证

- ✔ 画布尺寸：600×600 像素
- ✔ 蛇头颜色：蓝色 (#0066FF)
- ✔ 蛇身颜色：绿色 (#00CC00)
- ✔ 食物颜色：红色 (#FF3333)
- ✔ 方向键控制：上、下、左、右
- ✔ 计分系统：吃到食物分数+1
- ✔ 碰撞检测：撞墙或撞身体游戏结束
- ✔ 重新开始：游戏结束后显示"再来一局"按钮

- ☒ 最高分存储：使用 localStorage 持久化

四、智能体论文解读实验

4.1 智能体提示词

帮我解读C:\Users\34068\Desktop\双碳\里的1-s2.0-S1544612325013005-main这篇论文

4.2 论文文献条目

项目	内容
标题	Green bond issuance premium effect and investor incentive effect
作者	Yinhui Hao, Yan Zhou
单位	铜陵学院会计学院
期刊	Finance Research Letters
发表年份	2025年
DOI	https://doi.org/10.1016/j.frl.2025.108042
关键词	Green bonds, Premium effect, Incentive effect

4.3 论文解读节选

研究背景：

在"双碳"战略背景下，绿色债券作为绿色金融的重要工具，其发行溢价效应和投资者激励效应备受关注。本文以2017-2023年中国发行的7862只绿色债券和普通债券为样本，采用OLS模型进行实证分析。

核心发现：

1. **溢价效应存在：**绿色债券在一级市场存在显著溢价效应，信用利差显著低于普通债券，为企业提供了更经济的融资渠道。其中，绿色金融债券信用利差最低，市场吸引力最强。

2. **异质性特征**：不同类型绿色债券的溢价效应存在差异。绿色金融债券 > 绿色公司债券 > 绿色企业债券。绿色企业债券因允许50%资金用于非绿色项目，溢价效应较弱。
3. **投资者激励机制**：绿色债券在二级市场无法带来更高资本回报，但流动性显著优于普通债券，成为吸引投资者的重要因素。

研究贡献：

采用双视角分析框架（融资端+投资端），揭示了绿色债券的"双重激励机制"，为政策制定提供了细分依据。

五、Excel数据合并实验

5.1 合并日志摘要

合并时间：2026-06-24 22:53:05

源文件信息：

文件	格式	行数	列名映射
交易记录.xlsx	xlsx	150	customer_id→客户ID, amount→金额
产品购买.xlsx	xlsx	120	CustID→客户ID, 日期→交易日期
客户信息.xlsx	xlsx	100	无映射
客户反馈.xlsx	xlsx	80	name→姓名, 反馈日期→交易日期
营销活动.csv	csv	100	cust_id→客户ID, activity_date→交易日期
账户余额.xlsx	xlsx	100	客户编号→客户ID, transaction_date→交易日期

合并统计：

指标	数值
合并前行数	650
重复记录数	101
去重后行数	549

指标	数值
最终列数	27
唯一客户数	100

去重策略：基于"客户ID + 交易日期"复合键去重，保留首次出现记录

5.2 业绩预测报告

数据质量评估：

- ✔ 数据完整性：所有文件均成功读取
- ✔ 列名归一化：完成中英文映射和格式统一
- ✔ 数据类型统一：日期字段转换为datetime格式
- ⚠ 缺失值情况：不同业务表字段差异较大，部分字段缺失率较高（属正常现象）

业务洞察：

- 客户覆盖：**共涵盖100个唯一客户
- 时间跨度：**交易日期范围为2024年全年
- 数据来源分布：**交易记录最多（150条），客户反馈最少（80条）

建议：

- 后续分析可按业务类型分别处理各表数据
- 考虑将客户信息表与其他表进行关联查询而非纵向合并
- 对于缺失值较多的字段，建议根据业务需求进行填充或删除

六、实验心得

6.1 实验总结

本次实验涵盖了三个主要任务：贪吃蛇游戏开发、论文智能解读、Excel数据合并。通过Trae IDE的智能体辅助，显著提升了开发效率。以下是具体心得体会：

6.2 AI出错案例与修正

案例：命令行参数过长导致执行失败

问题描述：

在执行Excel合并脚本时，由于Python代码过长（超过32000字符限制），PowerShell无法直接执行，报错信息为：

```
Error: Command too long (32692 chars after encoding, limit: 32000)
```

原因分析：

PowerShell对单行命令长度有限制，直接在命令行执行长Python代码会触发此限制。

修正方法：

1. 将Python代码保存为独立的 .py 文件（merge_data.py）
2. 通过 python merge_data.py 命令执行脚本
3. 成功完成数据合并操作

经验教训：

- 当代码逻辑复杂或代码量较大时，应优先保存为文件再执行
- 熟悉开发环境的限制和最佳实践有助于提高效率
- Trae IDE虽然支持直接执行代码，但对于复杂任务，使用文件方式更可靠

6.3 实验收获

1. **智能体辅助开发**：Trae IDE的智能体功能能够快速理解自然语言需求并生成代码，极大提升了开发效率
2. **多任务协作**：在同一环境中完成游戏开发、论文解读、数据处理等多种任务，体现了一站式开发的优势
3. **问题解决能力**：通过遇到并解决实际问题（如命令长度限制），加深了对开发环境的理解
4. **文档规范**：智能体能够自动生成详细的日志和报告，提升了工作的规范性和可追溯性

6.4 改进建议

1. 建议在Trae IDE中增加代码长度检测功能，自动提示用户将长代码保存为文件
2. 增加更多模板化的实验报告生成功能，方便学生提交实验成果

3. 增强智能体对复杂任务的分段处理能力，避免一次性生成过长代码

报告完成时间：2026年6月24日

报告版本：v1.0

实验者：[姓名]

指导教师：[姓名]

实验心得

本次“A+B 综合实验”是我在 Trae IDE 智能体辅助开发环境中的首次完整实践，涵盖贪吃蛇游戏开发、智能体论文解读和 Excel 数据合并三项任务。通过这次实验，我对 AI 辅助编程的工作模式有了切身体会，也积累了宝贵的调试经验。

在贪吃蛇游戏开发中，我通过自然语言提示词向智能体描述了画布尺寸、颜色方案、方向键控制、计分与最高分持久化等需求。智能体迅速生成了完整的单文件 HTML 代码，游戏运行流畅，碰撞检测和 localStorage 存储均符合预期。这一过程让我感受到，清晰的提示词是高效协作的前提——我的提示词在清晰度和完整性上得分较高，这直接决定了生成代码的质量。

论文解读任务则展现了智能体在学术领域的实用价值。我仅提供了本地论文路径，智能体便自动提取了文献信息并生成了结构化的解读报告，精准概括了绿色债券溢价效应和投资者激励效应的核心发现，甚至区分了绿色金融债券、公司债券和企业债券的异质性。这为我今后阅读外文文献提供了高效的辅助工具。

Excel 数据合并是本次实验工作量最大的环节。智能体成功读取了六个不同格式的数据文件，完成了列名映射、日期格式统一和基于复合键的去重操作，最终合并为 549 行、27 列的完整数据集。然而，实验过程中也暴露了 AI 协作的局限——由于一次性生成的 Python 代码超过 32000 字符，PowerShell 报错“命令过长”。起初我有些困惑，但很快意识到应将长代码保存为独立的`.py`文件再执行。这个看似简单的问题让我深刻理解了开发环境的约束，也提醒我在复杂任务中应主动拆分步骤，而非依赖单次指令完成所有操作。

通过本次实验，我收获了以下经验：第一，智能体能大幅降低编码门槛，但

开发者仍需具备基本的调试和环境判断能力；第二，规范化的提示词和任务拆解是高效协作的关键；第三，实验日志和报告的自动生成有助于提升工作的可追溯性。当然，我也建议 Trae IDE 未来能内置代码长度检测提示，并增强对复杂任务的分段处理支持，让智能体更加“善解人意”。

总之，这次实验让我迈出了 AI 辅助开发的第一步。我不仅掌握了贪吃蛇游戏实现、论文智能解读和数据处理的基本流程，更重要的是学会了在遇到问题时冷静分析、灵活应对。这些能力将伴随我在今后的学习和研究中走得更远。